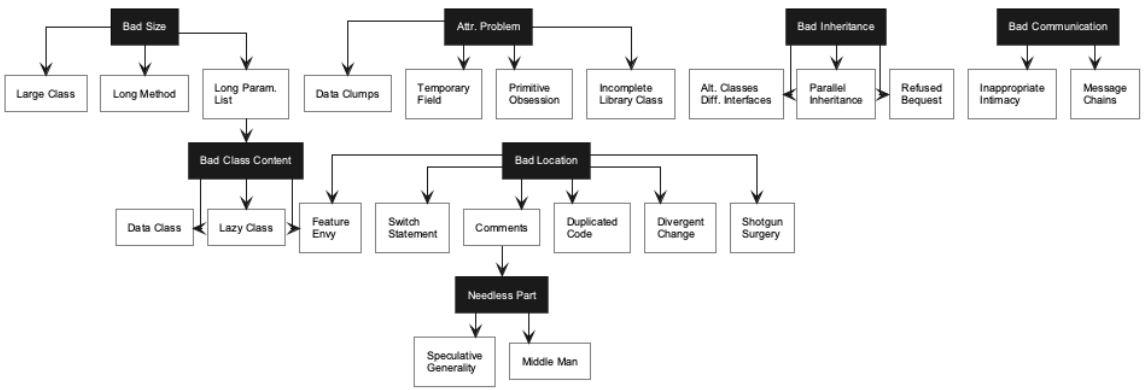
Dependency-Aware Code Smell Refactoring via Multi-Agent LLM System

Havriil Pietukhin, Ivan Polášek

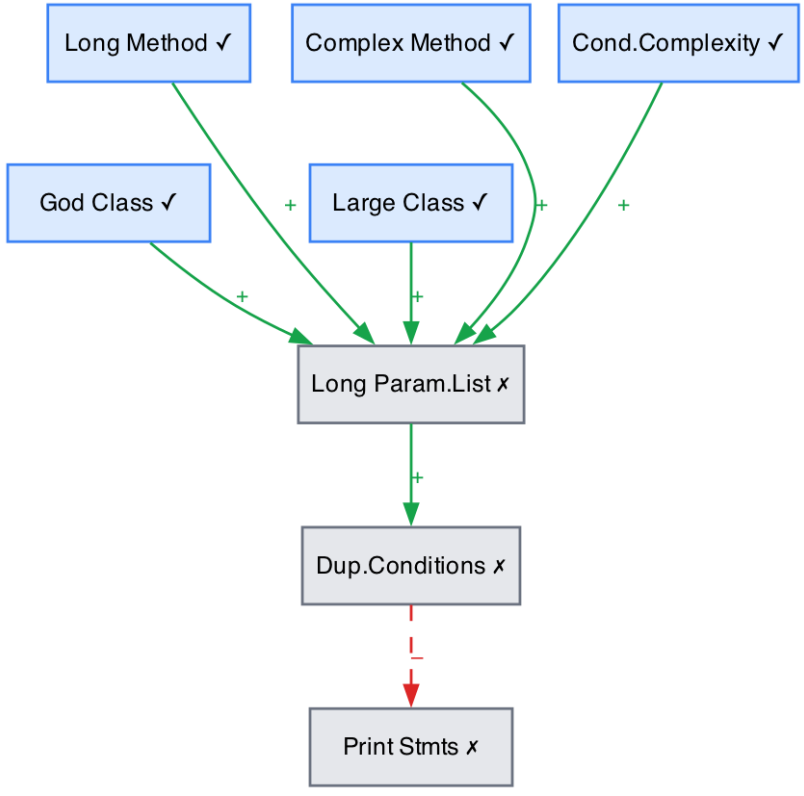
Problem & Goal

- Developers often fix code smells one at a time.
- In real projects, smells interact: fixing one can create another.
- LLMs can edit code, but the order of edits still matters.
- Goal: plan the refactoring sequence before asking an LLM agent to change the code.



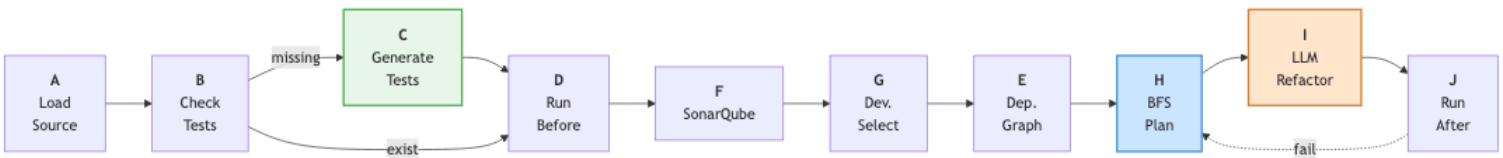
Dependency Model

- Each detected smell is a node in a graph.
- Green edges show cases where one refactoring may remove another smell.
- Red edges show cases where one refactoring may create another smell.
- The planner uses this graph to choose a safer order of actions.
- The graph can also store extra attributes, for example smell weight or project-specific priority.



System Pipeline

1. Load the Java project and check the build/test setup.
2. Detect smells with SonarQube.
3. Let the developer choose target smells.
4. Build the smell dependency graph.
5. Plan the order of refactorings.
6. Apply changes with an LLM agent and validate them with tests and code quality metrics.

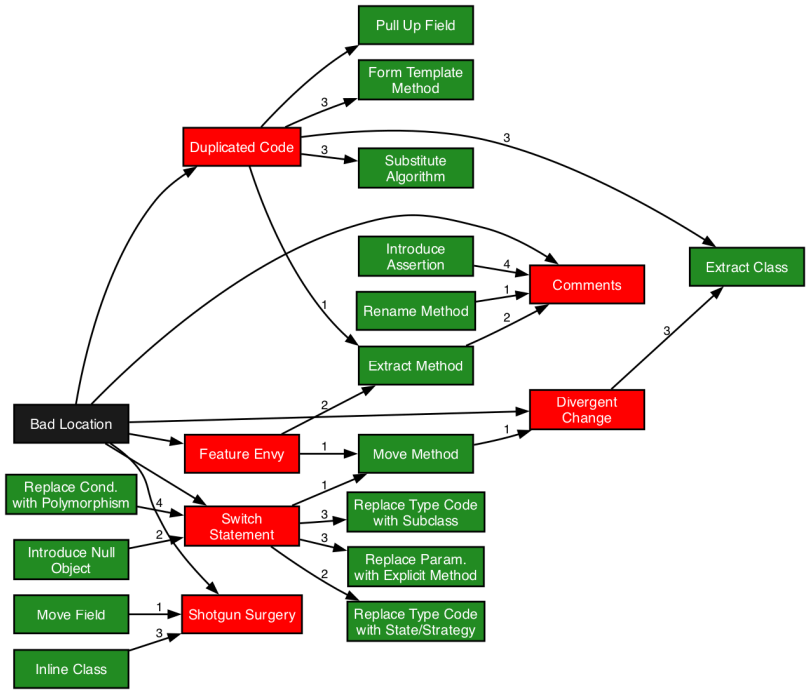


Planning Step

- Refactoring is treated as a search over the current set of active smells.
- Each action removes the selected smell, may remove related smells, and may introduce new ones.
- The greedy baseline chooses the locally highest-priority smell using severity, frequency, positive dependencies, and negative-dependency penalties.
- This can be unsafe: a high-priority God Class refactoring may fire negative edges and create extra Long Method or Inappropriate Intimacy smells.
- Best-First Search evaluates the next possible states and prefers the path with lower remaining smell severity.
- In the paper example, BeFS refactors Long Method before God Class, suppresses negative edges, and finishes in fewer steps.

Refactoring Agent

- The planner selects the next smell to handle.
- The LLM agent edits the affected Java class.
- The response is checked before the file is written.
- Tests run after each change; a failure triggers rollback and replanning.



Contribution & Future Work

- Multi-agent pipeline for Java refactoring: project setup, test baseline, SonarQube detection, developer selection, dependency graph construction, planning, LLM editing, testing, rollback, and replanning.
- Formal dependency graph for eight smell types, including positive edges that may resolve related smells and negative edges that may introduce new ones.
- Two planners are implemented: a greedy baseline and a Best-First Search planner that reasons over future smell states before invoking the LLM refactoring agent.
- Future work: build a labelled multi-smell dataset from RefactoringMiner 2.0 and SWE-Refactor to evaluate full refactoring sequences, not only first-step agreement.
- Future work: measure plan efficiency, new-smell introduction rate, and compile/test pass rate after each LLM patch.
- Future work: validate dependency rules against empirical smell trajectories and learn project-specific severity and negative-edge weights.